UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

**Mini Project Report**

**On**

# Hospital Patient Scheduling System


**Submitted by:**

Name: Nancy

Uid: 25MCA20253

Class: MCA (General)

Section: 25MCA-2-A

**Submitted to:**

Ms. Gagandeep Kaur

Assistant Professor

# Table of Content


| S.No. | Content | Page No. |
|---|---|---|
| **1** | Acknowledgement | 1 |
| **2** | Introduction | 2 |
| **3** | Objective | 3 |
| **4** | Technologies Used | 4 |
| **5** | Requirements and System Design | 5-6 |
| **6** | Source Code & Output | 7-13 |
| **7** | Functionalities | 13 |
| **8** | Conclusion | 14 |
| **9.** | Future Work/GitHub Link | 14 |

# Acknowledgement

I take this opportunity to express my sincere gratitude to everyone who has supported and guided me throughout the completion of this C++ project titled "Hospital Patient Scheduling System".

I am deeply thankful to Ms. Gagandeep Kaur for their valuable guidance, encouragement, and constant support during the development of this project. Their insightful suggestions, motivation, and expert advice have been instrumental in enhancing the quality and success of my work.

I would also like to extend my heartfelt thanks to Chandigarh University and the Department of Master of Computer Applications (MCA) for providing an excellent academic environment, essential resources, and continuous support that made this project possible.

My special thanks go to my classmates, friends, and family members for their unwavering encouragement, patience, and moral support throughout every stage of this case study.

Lastly, I wish to acknowledge the valuable assistance of various online learning platforms, GitHub repositories, AI tools such as Copilot, ChatGPT, Gemini, and the official C++ documentation, which greatly contributed to my learning and successful implementation of this project.

# Introduction

The Hospital Patient Scheduling System is a comprehensive C++ based console application developed to manage hospital operations efficiently. The system handles patient registration, doctor management, appointment scheduling, bed allocation, and discharge tracking through a modular, menu-driven interface.

This project implements a complete hospital management solution with persistent data storage using multiple file-based records (patients.txt, doctors.txt, appointments.txt, departments.txt, discharge.txt, and logs.txt). The application features real-time priority queue processing for emergency cases, department-wise bed management, and comprehensive activity logging with timestamps.

The system includes over 15 departments with pre-configured bed capacities, 45+ doctor profiles across specializations, and supports severity-based patient prioritization (1-10 scale). Users can register patients, assign doctors, schedule appointments, track bed occupancy, generate discharge summaries, and view detailed activity logs. The application also features a priority treatment queue that automatically sorts patients by severity, ensuring critical cases receive immediate attention.

# Objectives

The main objectives of the Hospital Patient Scheduling System are to provide an integrated platform for managing hospital workflows and to demonstrate various C++ programming concepts including Object-Oriented design, file handling, STL containers, and priority queues.

Specific Objectives:

1. **To implement comprehensive patient management** - Register new patients with complete medical records (blood group, severity, department assignment, clinical notes), update records, and manage discharges with summary reports.

2. **To provide doctor management system** - Maintain doctor profiles with specialization, qualification, experience, availability, and track patient load across 15+ departments.

3. **To enable appointment scheduling** - Schedule, view, and update appointments with time slot selection, purpose tracking, and status management (Scheduled/Completed/Cancelled).

4. **To manage hospital resources** - Track department-wise bed allocation (total/occupied/free beds), adjust occupancy during admissions and discharges.

5. **To implement severity-based prioritization** - Use priority queue data structure to automatically sort active patients by severity (1-10 scale) for emergency treatment ordering.

6. **To maintain comprehensive audit trails** - Timestamped activity logging for all operations (admissions, updates, discharges, appointments) with searchable log viewer.

7. **To generate operational reports** - Department status reports, hospital summary statistics, discharge records, and activity logs for administrative review.

# Technologies Used

The Hospital Patient Scheduling System is developed using C++ standard libraries and follows modern C++ programming practices. Each component plays a crucial role in handling different aspects of the project.

| S.No. | Technology | Purpose |
|---|---|---|
| 1 | C++17 | Core programming language |
| 2 | iostream | Console input/output operations |
| 3 | fstream | File handling for persistent data storage |
| 4 | vector | Dynamic array storage for patients, doctors, appointments |
| 5 | queue / priority_queue | Treatment queue with severity-based ordering |
| 6 | map | Department-doctor catalog mapping |
| 7 | string / sstream | String manipulation and data serialization |
| 8 | algorithm | Searching, sorting, and transformation operations |
| 9 | iomanip | Formatted table output |
| 10 | ctime | Timestamp generation for logs and dates |

# Requirements and System Design

## Hardware Requirements

| Component | Minimum |
|---|---|
| Processor | Intel Core i3 / AMD equivalent |
| RAM | 4 GB |
| Storage | 500 MB |
| Display | 1366×768 |

## Software Requirements

| Component | Specification |
|---|---|
| OS | Windows 10/11, macOS, Linux |
| Compiler | GCC (MinGW), Clang, or MSVC with C++17 support |
| IDE | VS Code / CodeBlocks / CLion / Dev-C++ |

## System Architecture (MVC)

| Layer | Component |
|---|---|
| View | Console-based menu system with formatted output |
| Controller | Patient, Doctor, Appointment, Reports controllers |
| Model | File-based data persistence (6 text files) |
| Utility | Logging, timestamp generation, input validation |

## Data File Structure

| File | Purpose | Format |
|---|---|---|
| patients.txt | Patient records | id\|name\|age\|gender\|phone\|bloodGroup\|severity\|dept\|doctor\|admitDate\|status\|notes |
| doctors.txt | Doctor records | id\|name\|spec\|dept\|qual\|phone\|exp\|avail\|patientCount |
| appointments.txt | Appointment records | id\|patientId\|patientName\|doctorId\|doctorName\|dept\|date\|time\|purpose\|status |
| departments.txt | Department info | name\|totalBeds\|occupiedBeds\|headDoctor\|floor\|phone |
| discharge.txt | Discharge summaries | patientId\|name\|dept\|doctor\|admitDate\|dischargeDate\|summary |
| logs.txt | Activity log | [timestamp] [action] detail |

# Source Code & Output

## 1. Data Structures Definition

```cpp
struct Patient {
    int    id;
    string name;
    int    age;
    string gender;
    string phone;
    string bloodGroup;
    int    severity;       // 1 (low) - 10 (critical)
    string department;
    string assignedDoctor;
    string admitDate;
    string status;         // Active | Discharged | Under Observation
    string notes;

    bool operator<(const Patient& o) const { return severity < o.severity; }
};

struct Doctor {
    int    id;
    string name;
    string specialization;
    string department;
    string qualification;
    string phone;
    int    experience;
    string availability;
    int    patientCount;
};

struct Appointment {
    int    id;
    int    patientId;
    string patientName;
    int    doctorId;
    string doctorName;
```

```
    string department;
    string date;
    string time;
    string purpose;
    string status;
};
```

**2. Priority Queue Implementation for Treatment Queue**

cpp

```
void displayPriorityQueue() {
    auto list = loadPatients();
    priority_queue<Patient> pq;

    for(auto& p : list)
        if(p.status != "Discharged")
            pq.push(p);

    cout << "\n  -- Priority Treatment Queue (Highest Severity First) --\n";
    int rank = 1;
    while(!pq.empty()) {
        cout << "  " << rank++ << " ";
        printPatientRow(pq.top());
        pq.pop();
    }
}
```

**3. File I/O with Serialization**

cpp

```
string serializePatient(const Patient& p) {
    return to_string(p.id) + "|" + p.name + "|" + to_string(p.age) + "|" +
           p.gender + "|" + p.phone + "|" + p.bloodGroup + "|" +
           to_string(p.severity) + "|" + p.department + "|" +
           p.assignedDoctor + "|" + p.admitDate + "|" + p.status + "|" + p.notes;
}

vector<Patient> loadPatients() {
    vector<Patient> list;
    ifstream f(FILE_PATIENTS);
    if(!f) return list;

    string line;
    while(getline(f, line)) {
```

```
        if(line.empty()) continue;
        Patient p;
        if(deserializePatient(line, p))
            list.push_back(p);
    }



    return list;
}
```

**4. Department and Doctor Catalog (Pre-configured Data)**

cpp

```
const vector<DeptInfo> DEPT_CATALOG = {
    {"Cardiology", "3rd Floor", "ext-301", 30},
    {"Neurology",  "4th Floor", "ext-401", 25},
    {"Orthopedics", "2nd Floor", "ext-201", 35},
    // ... 15 departments total
};

const map<string, vector<tuple<string,string,int>>> DOCTOR_CATALOG = {
    {"Cardiology", {{"Dr. Arjun Sharma","MD, DM Cardiology",18},
                    {"Dr. Priya Nair","MBBS, MD, FACC",12}}},
    // ... doctors for each department
};
```

**5. Activity Logging System**

cpp

```
void appendLog(const string& action, const string& detail) {
    ofstream f(FILE_LOGS, ios::app);
    f << "[" << currentTimestamp() << "] [" << action << "] " << detail << "\n";
}

string currentTimestamp() {
    time_t now = time(nullptr);
    char buf[20];
    strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", localtime(&now));
    return string(buf);
}
```

# Output:-

## Main Menu

The Main Menu displays the system header, lists all data files used, and provides navigation to four main modules: Patient Management, Doctor Management, Appointment Scheduling, and Reports & Logs.

```
==========================================================================
          HOSPITAL PATIENT SCHEDULING SYSTEM  v2.0
==========================================================================
  Data files:
    patients.txt       doctors.txt        appointments.txt
    departments.txt    discharge.txt       logs.txt
--------------------------------------------------------------------------
  [1]  Patient Management
  [2]  Doctor Management
  [3]  Appointment Scheduling
  [4]  Reports & Logs
  [0]  Exit
--------------------------------------------------------------------------
  Choice:
```

## Patient Registration

The Patient Registration form collects comprehensive patient information including name, age, gender, phone, blood group, severity (1-10), department selection from 15 options, doctor assignment from department-specific catalogs, and clinical notes.

```
==========================================================================
  PATIENT MANAGEMENT
==========================================================================
  [1] Register New Patient
  [2] Update Patient Record
  [3] Discharge Patient
  [4] Search Patients
  [5] Priority Treatment Queue
  [6] View All Patients
  [0] Back
--------------------------------------------------------------------------
  Choice:
```

patients.txt

```
1|Nancy|21|Female|7988122236|O+|4|General Medicine|Dr. Anjali Mehta|2026-04-09|Active|Cough and cold
2|Lucky|23|Female|8765678765|AB+|8|ICU|Dr. Rahul Mishra|2026-04-09|Discharged|Heart Attack
3|Aman|21|Male|6578987656|B-|1|Dermatology|Dr. Nisha Bakshi|2026-04-09|Active|Skin issue
```

discharge.txt

```
2|Lucky|ICU|Dr. Rahul Mishra|2026-04-09|2026-04-09|Treatment done
```

**Priority Treatment Queue**

The Priority Queue displays all active patients sorted by severity (highest first), ensuring critical cases receive immediate attention. Each row shows ID, name, severity score, department, assigned doctor, and current status.

```
======================================================================
  Priority Treatment Queue  (Highest Severity First)
======================================================================
----------------------------------------------------------------------
 ID   Name                 Age  Gender  Sev   Department        Doctor                 Status
----------------------------------------------------------------------
  #1  2    Lucky                23   Female  8     ICU               Dr. Rahul Mishra       Active
  #2  1    Nancy                21   Female  4     General Medicine  Dr. Anjali Mehta       Active
  #3  3    Aman                 21   Male    1     Dermatology       Dr. Nisha Bakshi       Active
```

**Department Status Report**

The Department Status report shows real-time bed availability across all departments including total beds, occupied beds, free beds, head doctor, and floor location.

departments.txt

```
Cardiology|30|1|Dr. Arjun Sharma|3rd Floor|ext-301
Neurology|25|0|Dr. Kavita Rao|4th Floor|ext-401
Orthopedics|35|0|Dr. Vikram Patel|2nd Floor|ext-201
Pediatrics|40|0|Dr. Sunita Agarwal|1st Floor|ext-101
Oncology|20|0|Dr. Rajiv Bose|5th Floor|ext-501
Emergency|50|0|Dr. Deepak Tiwari|Ground|ext-001
General Medicine|45|3|Dr. Ramesh Iyer|1st Floor|ext-102
Gynecology|30|0|Dr. Lakshmi Reddy|2nd Floor|ext-202
Dermatology|15|1|Dr. Aisha Khan|3rd Floor|ext-302
Psychiatry|20|0|Dr. Prashant Bhat|4th Floor|ext-402
Ophthalmology|10|0|Dr. Samir Kulkarni|3rd Floor|ext-303
ENT|15|0|Dr. Arun Saxena|2nd Floor|ext-203
Urology|20|0|Dr. Sunil Chandra|4th Floor|ext-403
Pulmonology|25|0|Dr. Girish Naik|5th Floor|ext-502
ICU|15|0|Dr. Rahul Mishra|Ground|ext-002
```

doctors.txt

```
1|Sunil Kashyap|Maternal-Fetal Medicine (MFM)|Gynecology|MBBS,MD,DGO|9876787656|4|Evening|0
```

## Appointment Scheduling

The Appointment Scheduler allows users to select patient ID, choose department and doctor, pick date and time slot from predefined options, and record purpose before confirming the appointment.

appointments.txt

```
1|3|Aman|0|Dr. Sunita Agarwal|Pediatrics||10:00|Tests|Scheduled
```

## Activity Logs

The Activity Logs viewer displays timestamped records of all system operations including patient admissions, discharges, updates, and appointments with search/filter functionality.

logs.txt

```
[2026-04-09 15:46:12] [ADMITTED] ID:1 | Nancy | Dept:General Medicine | Dr:Dr. Anjali Mehta | Severity:4
[2026-04-09 15:48:13] [ADMITTED] ID:2 | Lucky | Dept:Emergency | Dr:Dr. Deepak Tiwari | Severity:8
[2026-04-09 15:49:24] [ADMITTED] ID:3 | Aman | Dept:Dermatology | Dr:Dr. Nisha Bakshi | Severity:1
[2026-04-09 15:50:30] [UPDATED] ID:2 | Lucky | Dept:ICU | Sev:8
[2026-04-09 15:52:02] [DISCHARGED] ID:2 | Lucky | ICU
[2026-04-09 21:32:08] [APPOINTMENT] ApptID:1 | Patient:Aman | Dr:Dr. Sunita Agarwal |  10:00
[2026-04-09 21:39:10] [DOCTOR_ADDED] ID:1 | Sunil Kashyap | Gynecology
```

# Functionalities

| Page/Module | Functionalities |
|---|---|
| Patient Management | Patient Registration, Update Records, Discharge Processing, Search (ID/Name/Dept/Doctor/Status), Priority Treatment Queue, View All Patients |
| Doctor Management | Add Doctor Records, View All Doctors, View Doctors by Department, Track Patient Load |
| Appointment Scheduling | Schedule Appointments, View Appointments (All/By Patient/By Doctor/Scheduled), Update Appointment Status (Scheduled/Completed/Cancelled) |
| Reports & Logs | Activity Logs Viewer (with keyword filter), Discharge Records, Department Status (Bed Availability), Hospital Summary Report |

# Conclusion

The Hospital Patient Scheduling System successfully demonstrates the practical application of C++ programming concepts in developing a comprehensive healthcare management solution. The system integrates file-based persistence, STL containers (vector, map, priority_queue), and modular design patterns to deliver a functional hospital administration tool. Key achievements include severity-based patient prioritization using priority queues, complete CRUD operations for patients and doctors, bed allocation tracking across 15 departments, timestamped activity logging, and discharge management with permanent record keeping. Overall, the system provides an excellent educational tool for understanding C++ file handling, data structures, and real-world application development.

# Future Work

The Hospital Patient Scheduling System can be extended with the following features:

- **Graphical User Interface** - Migrate from console-based to GUI using Qt or web-based frontend
- **Database Integration** - Replace text files with SQLite/MySQL database for better data integrity
- **User Authentication** - Implement role-based access control (Admin, Doctor, Receptionist, Nurse) with login system
- **Prescription Management** - Add medication tracking, prescription generation, and pharmacy integration
- **Billing System** - Integrate complete billing module with insurance claim processing
- **Online Appointment Portal** - Allow patients to book appointments via web/mobile interface
- **SMS/Email Notifications** - Automated reminders for appointments and discharge summaries

# GitHub Link

https://github.com/Nancy5848/Hospital_Patient_Scheduling_System